

5.2 AP Mode ESP32

ESP32 AP Mode

Note: This tutorial uses the ESP32 development board to create a WiFi hotspot (AP mode), which is an important step in learning advanced ESP32 wireless communication. Please make sure your ESP32 has the correct drivers and board core support installed.

Related Knowledge

What is AP Mode?

1

AP Mode (Access Point) means the ESP32 itself becomes a WiFi hotspot, similar to your home wireless router. Other devices (phones, computers, other ESP32 boards) can connect to the ESP32 just like connecting to a regular WiFi network, enabling data exchange with it.

Simple analogy: Just like turning on your phone's "Personal Hotspot", when the ESP32 enables AP mode, it becomes a miniature "wireless router" that other devices can search for and connect to via its WiFi signal.

STA Mode vs AP Mode

ESP32's WiFi has two main working modes. Beginners need to clearly understand the differences:

Mode	Full Name	Analogy	Description
STA	Station	Phone connecting to router	ESP32 acts as a "client" connecting to an existing WiFi hotspot
AP	Access Point	The router itself	ESP32 creates its own WiFi hotspot for other devices to connect to
AP+STA	Both modes simultaneously	Both a router and a client	Can connect to others and be connected to by others



This tutorial uses **AP mode**, where the ESP32 creates its own WiFi hotspot for other devices to connect to.

ESP32 Development Board

ESP32 is an IoT development board developed by Espressif:

- ▶ Built-in WiFi and Bluetooth, making it a mainstream choice for IoT development
- ▶ Dual-core processor, 240MHz main frequency, far outperforming the traditional Arduino UNO
- ▶ Supports STA/AP mode switching, flexibly adapting to various network scenarios
- ▶ In this project, the ESP32 acts as a "WiFi hotspot"

Three Stages of AP Mode Startup

1. **Configure Static IP:** Set the IP address of the ESP32 hotspot (default 192.168.4.1)
2. **Start AP:** Create a WiFi hotspot named `ESP32_AP`
3. **Wait for Clients:** Loop-check how many devices are connected to the hotspot and indicate the status via an LED

IP Address and MAC Address

- ▶ **IP Address:** The device's "house number" in the network. In AP mode, the ESP32's default IP is `192.168.4.1`
- ▶ **MAC Address:** The device's "physical ID card," globally unique (e.g., `24:6F:28:A4:B2:C1`), determined by hardware
- ▶ **Subnet Mask:** Determines the network range; commonly `255.255.255.0`
- ▶ **Gateway:** The network exit address; in AP mode, it is usually the same as the ESP32's own IP

WiFi Channel

WiFi channels are the frequency bands used for wireless signal transmission:

- ▶ 2.4GHz WiFi has 1~13 channels
- ▶ Among them, channels **1, 6, and 11** do not overlap with each other and are commonly used "non-interfering channels"
- ▶ This project uses channel **6**
- ▶ The channel is specified by the third parameter of `softAP()`

Client (Station) Management

In AP mode, the ESP32 can manage devices connected to it (called Stations):

- Supports up to **4** simultaneous client connections (modifiable in [softAP\(\)](#) , maximum 10)
- Can query the current number of connected clients
- Clients are automatically assigned IPs starting from [192.168.4.2](#)

[Back to Elegoo](#)

Key Functions

AP Mode Core Functions

Function	Description	Role in This Project
WiFi.softAPConfig(ip, gateway, subnet)	Configure the static IP for AP	Set the ESP32's AP IP to 192.168.4.1
WiFi.softAP(ssid, password, channel, hidden, max)	Start AP hotspot	Create a hotspot named ESP32_AP
WiFi.softAPIP()	Get the AP's IP address	Get the current AP's IP
WiFi.softAPmacAddress()	Get the AP's MAC address	Print the ESP32's hardware address
WiFi.softAPgetStationNum()	Get the number of connected clients	Periodically print the current connection count
WiFi.channel()	Get the current WiFi channel	Print the channel number in use

[WiFi.softAP\(\)](#) Parameter Details

1 | `WiFi.softAP(ssid, password, channel, hidden, max_connection);`

Parameter	Description	Value Range
ssid	Hotspot name	Any string (recommended: English + numbers)
password	Hotspot password	Empty ("") for open network; otherwise at least 8 characters
channel	WiFi channel	1~13, default 1
hidden	Whether to hide the SSID	0 = visible, 1 = hidden
max_connection	Maximum connections	Default 4, maximum 10



This project uses: `WiFi.softAP(AP_SSID, AP_PASSWORD, 6, 0, 4)` , which means channel 6, SSID visible, maximum 4 clients

Time-Related Functions

Function	Description	Role in This Project
millis()	Returns milliseconds since boot	Implement 3-second periodic client count check
delay(ms)	Blocking delay	Serial stabilization, loop throttling, etc.

Complete Code Analysis

 [DOWNLOAD AP Mode](#)

1. Header File and Variable Definitions

```
1 | #include <WiFi.h>
2 |
3 | //===== AP Configuration =====
4 | const char* AP_SSID = "ESP32_AP"; // Hotspot name
5 | const char* AP_PASSWORD = "12345678"; // Min 8 characters, or "" for open network
6 | #define LED_BUILTIN 2
```

```

7 |
8 | // Set static IP for the AP (clients will get IPs in this range)
9 | IPAddress local_IP(192, 168, 4, 1);
10 | IPAddress gateway(192, 168, 4, 1);
11 | IPAddress subnet(255, 255, 255, 0);
12 |
13 | // ===== Timing =====
14 | unsigned long lastClientCheck = 0;
15 | const unsigned long CLIENT_CHECK_INTERVAL = 3000; // Check every 3s

```

Code Explanation:

- ▶ `#include <WiFi.h>` : ESP32 WiFi core library, providing AP/STA related functions
- ▶ `AP_SSID` : Hotspot name, this is what you see when searching for WiFi on your phone
- ▶ `AP_PASSWORD` : Hotspot password, **must be at least 8 characters**; leave empty for an open network (no password)
- ▶ `LED_BUILTIN 2` : The on-board LED pin number of the ESP32 dev board (usually GPIO2), used as a connection status indicator
- ▶ `local_IP` : The ESP32's IP address in AP mode, default subnet is `192.168.4.x`
- ▶ `gateway` : Gateway address, usually the same as the device's own IP in AP mode
- ▶ `subnet` : Subnet mask, `255.255.255.0` means the same subnet can accommodate 254 devices
- ▶ `lastClientCheck` / `CLIENT_CHECK_INTERVAL` : Non-blocking timing variables, check the client count every 3 seconds

Modification Tip: You can change `AP_SSID` and `AP_PASSWORD` to your preferred name and password.

2. setup() Initialization Function

`setup()` runs only once after the ESP32 powers on, and is responsible for completing all AP initialization.

2.1 Serial Initialization

```

1 | Serial.begin(115200);
2 | delay(1000);
3 |
4 | Serial.println("\n=====");
5 | Serial.println("    ESP32 AP Mode");
6 | Serial.println("=====");

```

Step Analysis:

- ▶ `Serial.begin(115200)` : Initialize the serial port at a baud rate of 115200, used to output debug information to the computer
- ▶ `delay(1000)` : Wait 1 second for the serial port to stabilize, avoiding loss of early output
- ▶ `Serial.println()` : Print a welcome banner to confirm the program is running normally

2.2 Step 1: Configure Static IP

```

1 | // ----- Step 1: Configure static IP -----
2 | Serial.println("[1/3] Configuring static IP...");
3 | if (!WiFi.softAPConfig(local_IP, gateway, subnet)) {
4 |   Serial.println("  Failed to configure AP IP!");
5 | } else {
6 |   Serial.print("  AP IP: ");
7 |   Serial.println(local_IP);
8 | }

```

Code Analysis:

- ▶ `WiFi.softAPConfig(local_IP, gateway, subnet)` : Set a static IP for the AP
- ▶ This step **must be called before** `softAP()`, otherwise the IP will not take effect
- ▶ After successful configuration, the ESP32's AP IP is fixed at `192.168.4.1`, and clients will be assigned starting from `192.168.4.2`
- ▶ If configuration fails, an error message is printed but the program continues (using the default IP)



Why configure a static IP? By default, the ESP32's AP IP is also 192.168.4.1, but explicit configuration ensures the IP does not change with firmware version changes, making it easier for clients to connect.

2.3 Step 2: Start AP Hotspot

```

1 //----- Step 2: Start AP mode -----
2 Serial.println("[2/3] Starting AP mode...");
3
4 bool result = WiFi.softAP(AP_SSID, AP_PASSWORD, 6, 0, 4);
5
6 if (!result) {
7     Serial.println(" Failed to start AP!");
8     return;
9 }
10
11 Serial.print(" SSID : ");
12 Serial.println(AP_SSID);
13 Serial.print(" Password : ");
14 Serial.println(AP_PASSWORD);
15 Serial.print(" Channel : ");
16 Serial.println(WiFi.channel());
17 Serial.print(" AP MAC : ");
18 Serial.println(WiFi.softAPmacAddress());

```

Code Analysis:

- `WiFi.softAP(AP_SSID, AP_PASSWORD, 6, 0, 4)` : Start AP, parameter meanings:
 - SSID: `ESP32_AP`
 - Password: `12345678`
 - Channel: `6` (1~13 available, commonly 1, 6, 11 to avoid interference)
 - Hidden: `0` (SSID visible, can be searched by phone)
 - Max connections: `4`
- Return value `result` : `true` means startup successful, `false` means failed
- On failure, `return` is used to directly end `setup()` to avoid subsequent invalid operations
- On success, print SSID, password, channel, MAC address and other information for verification



Channel Selection Tip: 2.4GHz WiFi has channels 1~13, among which 1, 6, and 11 do not overlap with each other and are commonly used "non-interfering channels".

2.4 Step 3: Ready Prompt and Pin Initialization

```

1 //----- Step 3: Ready -----
2 Serial.println("[3/3] Setup complete. Waiting for clients...");
3 Serial.println("-----");
4 Serial.println("Connect to this AP with WiFi:");
5 Serial.print(" SSID: ");
6 Serial.println(AP_SSID);
7 Serial.print(" IP : ");
8 Serial.println(local_IP);
9 Serial.println("-----\n");
10
11 pinMode(LED_BUILTIN, OUTPUT);

```

Code Analysis:

- Print a complete connection guide telling the user: connect to the WiFi hotspot named `ESP32_AP` with a phone/computer
- `pinMode(LED_BUILTIN, OUTPUT)` : Set the on-board LED pin to output mode for subsequent connection status indication

3. `loop()` Main Loop Function

`loop()` is called repeatedly after `setup()` finishes. It is the "heartbeat" of the program. The main loop of this project is responsible for **periodically checking the client count and indicating the status via LED**.

```

1 void loop() {
2     //----- Periodic WiFi client report -----
3     unsigned long now = millis();
4     if (now - lastClientCheck >= CLIENT_CHECK_INTERVAL) {
5         lastClientCheck = now;
6
7         int numClients = WiFi.softAPgetStationNum();
8         Serial.print("[WiFi] Connected stations: ");
9

```

```

7 | Serial.println(numClients);
10 |
11 | //LED indicator: on if any client connected
12 | if (numClients > 0) {
13 |     digitalWrite(LED_BUILTIN, HIGH);
14 | } else {
15 |     digitalWrite(LED_BUILTIN, LOW);
16 | }
17 | }
18 |
19 | delay(10);
20 | }

```

[Back to Elegoo](#)

Code Analysis:

Step 1: Non-Blocking Timing

```

1 | unsigned long now = millis();
2 | if (now - lastClientCheck >= CLIENT_CHECK_INTERVAL) {
3 |     lastClientCheck = now;

```

- `millis()` : Returns the number of milliseconds since the ESP32 booted (overflows after approximately 50 days)
- `now - lastClientCheck` : Calculates how long it has been since the last check
- `CLIENT_CHECK_INTERVAL` (3000ms): Only execute a check if more than 3 seconds have passed, avoiding frequent queries



Why use `millis()` instead of `delay()` ? Because `delay()` blocks the entire program, while `millis()` only records a timestamp and does not affect other logic execution. This is an important "non-blocking" programming paradigm.

Step 2: Query Client Count

```

1 | int numClients = WiFi.softAPgetStationNum();
2 | Serial.print("[WiFi] Connected stations: ");
3 | Serial.println(numClients);

```

- `WiFi.softAPgetStationNum()` : Get the current number of clients connected to the AP
- Print via serial port to let users understand the hotspot connection status in real time

Step 3: LED Status Indication

```

1 | if (numClients > 0) {
2 |     digitalWrite(LED_BUILTIN, HIGH); // Clients connected, LED on
3 | } else {
4 |     digitalWrite(LED_BUILTIN, LOW); // No clients, LED off
5 | }

```

- Visually reflect the connection status through the on-board LED
- LED stays on when a device is connected, turns off when there is no connection

Step 4: Loop Throttling

```

1 | delay(10);

```

- Pause 10ms in each loop to avoid CPU overload
- Also leaves processing time for the hardware and WiFi module

Main Loop Flowchart:

```

1 | loop() starts
2 | |
3 | |— Less than 3s since last check? — YES —→ delay(10), enter next round
4 | |
5 | |— 3s or more since last check
6 | |

```

```

7 | |— Query current client count and print
8 | |
9 | |— Has clients? — YES —> LED on
10 | |
11 | |— No clients —> LED off
12 | |
13 | |— delay(10), enter next round

```

[Back to Elegoo](#)

Operation Steps Guide

Step 1: Modify Hotspot Configuration (Optional)

Open [ap_mode.ino](#) and modify the following content as needed:

```

1 | const char* AP_SSID = "MyESP32"; // Change to your desired hotspot name
2 | const char* AP_PASSWORD = "mypassword"; // At least 8 characters, or "" for no password

```



Note: SSID is recommended to use only English letters and numbers to avoid encoding issues with Chinese or special characters.

Step 2: Connect Hardware

1. Connect the ESP32 development board to your computer using a USB data cable
2. Make sure the cable supports data transfer (some cables are for charging only)

Step 3: Upload the Program

1. Open the Arduino IDE
2. Select the board: **Tools → Board → ESP32 Dev Module** (or your specific model)
3. Select the port: **Tools → Port → the correct COM port**
4. Click the **Upload** button (→ arrow icon)
5. Wait for compiling and uploading to complete (may take a few minutes the first time)

Step 4: View Serial Output

1. Open the Serial Monitor (magnifying glass icon in the toolbar)
2. Set the baud rate to **115200** (bottom right corner)
3. You should see output similar to the following:

```

1 | =====
2 |     ESP32 AP Mode
3 | =====
4 | [1/3] Configuring static IP...
5 |     AP IP: 192.168.4.1
6 | [2/3] Starting AP mode...
7 |     SSID : ESP32_AP
8 |     Password : 12345678
9 |     Channel : 6
10 |     AP MAC : 24:6F:28:A4:B2:C1
11 | [3/3] Setup complete. Waiting for clients...
12 | -----
13 | Connect to this AP with WiFi:
14 |     SSID: ESP32_AP
15 |     IP : 192.168.4.1
16 | -----
17 |
18 | [WiFi] Connected stations: 0

```

Step 5: Connect to the ESP32 Hotspot

1. Open the WiFi list on your phone/computer
2. Find the WiFi named [ESP32_AP](#)
3. Enter password [12345678](#) to connect
4. After successful connection, the phone will prompt "Connected, but no internet access" — this is normal, because we are only establishing a local network
5. At this point, observe the Serial Monitor; it should show [Connected stations: 1](#) , and the on-board LED will also light up

Expected Serial Output

When Startup Succeeds

[Back to Elegoo](#)

```
1 | =====
2 |     ESP32 AP Mode
3 | =====
4 | [1/3] Configuring static IP...
5 |     AP IP: 192.168.4.1
6 | [2/3] Starting AP mode...
7 |     SSID : ESP32_AP
8 |     Password : 12345678
9 |     Channel : 6
10 |     AP MAC : 24:6F:28:A4:B2:C1
11 | [3/3] Setup complete. Waiting for clients...
12 | -----
13 | Connect to this AP with WiFi:
14 |     SSID: ESP32_AP
15 |     IP : 192.168.4.1
16 | -----
17 |
18 | [WiFi] Connected stations: 0
```

When a Device Connects

```
1 | [WiFi] Connected stations: 1
2 | [WiFi] Connected stations: 1
3 | [WiFi] Connected stations: 2
```

At this point, the on-board LED will light up.

When a Device Disconnects

```
1 | [WiFi] Connected stations: 1
2 | [WiFi] Connected stations: 0
```

At this point, the on-board LED turns off.

When AP Startup Fails

```
1 | [2/3] Starting AP mode...
2 | Failed to start AP!
```



This situation usually occurs when the password is less than 8 characters or the SSID contains illegal characters.

FAQ

Common Questions

Q1: The Serial Monitor shows garbled characters?

A: Please make sure the baud rate of the Serial Monitor is set to **115200** (bottom right dropdown).

Q2: My phone cannot find the ESP32_AP hotspot?

A: Please check in order:

- ▶ Does the serial show **Failed to start AP!** ? (If so, the password needs to be ≥ 8 characters)
- ▶ Is the password entered correctly?
- ▶ Does the phone support 2.4GHz WiFi? (All phones do, but some older laptops may only search for 5G)
- ▶ Is the SSID hidden? (In this project, **hidden=0** , visible by default)

Q3: The phone prompts "No internet access" after connecting?

A: This is normal. The ESP32's AP mode only establishes a local network and has no routing function, so it cannot access the internet. If you want to access the internet, you need to use STA mode or AP+STA hybrid mode.

Q4: The LED never lights up?

A: Please check:

- ▶ Is any device connected to the ESP32_AP hotspot? (LED only lights up when there is a client) [Back to Elegoo](#)
- ▶ Is your development board's LED pin GPIO2? Different boards may have different pins (such as GPIO0, GPIO4); please modify `#define LED_BUILTIN 2`.

Q5: I get kicked offline after connecting multiple devices?

A: The ESP32 AP mode supports a maximum of 4 clients by default. If you need more, you can modify the last parameter of `softAP()` (maximum 10):

```
1 | WiFi.softAP(AP_SSID, AP_PASSWORD, 6, 0, 10); // Up to 10 clients
```

Q6: Can the AP IP address be changed to something else?

A: Yes, just modify `IPAddress local_IP(...)`, for example `192.168.10.1`. Clients will automatically obtain IPs from the same subnet.

Q7: Can I make the hotspot passwordless?

A: Yes, just leave the password parameter empty:

```
1 | WiFi.softAP(AP_SSID, "", 6, 0, 4); // Open network
```

Troubleshooting Table

Problem	Possible Cause	Solution
Garbled serial output	Wrong baud rate	Change to 115200
Cannot upload program	Port/driver issue	Check COM port and CH340/CP2102 driver
AP startup fails	Password < 8 characters	Password must be at least 8 characters, or leave empty ""
AP startup fails	SSID contains special characters	Use English + numbers
Phone cannot find hotspot	SSID is hidden	Check if <code>hidden</code> parameter is 0
Phone cannot connect	Wrong password	Verify password case sensitivity
LED not lit	No client connected	First connect the hotspot with a phone
LED not lit	Wrong pin definition	Modify <code>LED_BUILTIN</code> to match your board's pin
Frequent disconnections	Insufficient power	Use an independent 5V power supply (VIN pin)
Connection limit reached	Exceeded max_connection	Increase the 5th parameter of <code>softAP</code>

Modification Suggestions

1. Modify Hotspot Name and Password

```
1 | const char* AP_SSID = "MyHomeIoT"; // Change to a distinctive name
2 | const char* AP_PASSWORD = "88888888"; // Change to your own password (≥8 characters)
```

2. Create an Open Hotspot (No Password)

```
1 | WiFi.softAP(AP_SSID, "", 6, 0, 4); // Empty password means open network
```

3. Hide SSID (Make the Hotspot Unsearchable)

```
1 | WiFi.softAP(AP_SSID, AP_PASSWORD, 6, 1, 4); // hidden=1
2 | // Clients need to manually enter the SSID to connect
```

4. Increase Maximum Connections

```
1 | WiFi.softAP(AP_SSID, AP_PASSWORD, 6, 0, 10); // Up to 10 clients
```

5. Change WiFi Channel

[Back to Elegoo](#)

```
1 | WiFi.softAP(AP_SSID, AP_PASSWORD, 1, 0, 4); // Change to channel 1
2 | WiFi.softAP(AP_SSID, AP_PASSWORD, 11, 0, 4); // Change to channel 11
```



1, 6, and 11 are three commonly used non-interfering channels. You can choose according to environmental interference.

6. Change Detection Cycle

```
1 | const unsigned long CLIENT_CHECK_INTERVAL = 1000; // Change to check once per second
2 | const unsigned long CLIENT_CHECK_INTERVAL = 5000; // Change to check once every 5 seconds
```

7. Change AP IP Address

```
1 | IPAddress local_IP(192, 168, 10, 1); // Change to 192.168.10.1
2 | IPAddress gateway(192, 168, 10, 1);
3 | IPAddress subnet(255, 255, 255, 0);
```

8. Switch to AP+STA Hybrid Mode

```
1 | WiFi.mode(WIFI_AP_STA); // Enable both AP and STA simultaneously
2 | WiFi.softAP(AP_SSID, AP_PASSWORD); // Start AP
3 | WiFi.begin("YourHomeWiFi", "password"); // Connect to router simultaneously
```

Notes

Note 1: Password Length Limit

The AP mode password **must be at least 8 characters**, otherwise `softAP()` will return `false`. If you want an open network (no password), please leave the password parameter as `""`.

Note 2: 2.4GHz Limit

ESP32 only supports **2.4GHz WiFi**, not 5GHz. However, since AP mode is the ESP32's own signal transmission, you do not need to worry about the router's frequency band. All devices supporting 2.4GHz can connect.

Note 3: Power Supply Requirements

The ESP32's WiFi function has high power consumption (peak can reach over 300mA). If restarts occur when connecting multiple clients, it is recommended to use an independent 5V power supply through the VIN pin rather than relying solely on USB.

Note 4: Client Count Limit

ESP32's AP mode **supports up to 4 clients by default**, and can be set to a maximum of 10 through the `max_connection` parameter. New clients cannot connect after exceeding the limit.

Note 5: No Internet Access

Devices connected to the ESP32 AP hotspot **cannot access the internet** (because the ESP32 has no routing function). The phone will prompt "Connected but no internet access", which is normal. This project only establishes a local network between the ESP32 and clients.

Note 6: LED Pin Differences

The on-board LED pin may vary across different ESP32 development boards. This project uses `GPIO2` (the most common model). If your board's LED is on another pin (such as GPIO0, GPIO4, GPIO33), please modify `#define LED_BUILTIN 2`.

Note 7: IP Configuration Order

`WiFi.softAPConfig()` **must be called before** `WiFi.softAP()`, otherwise the static IP configuration will not take effect and the ESP32 will use the default IP `192.168.4.1`.

←WiFi Connet

WiFi Stability→